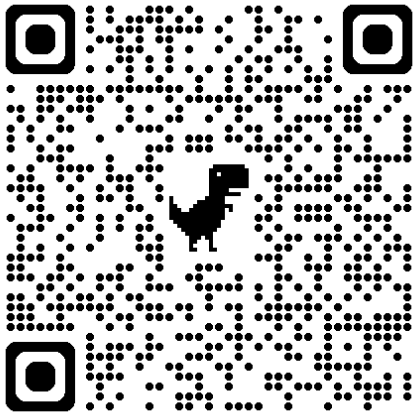
EC523 Fall 2026
Course Project Pitch

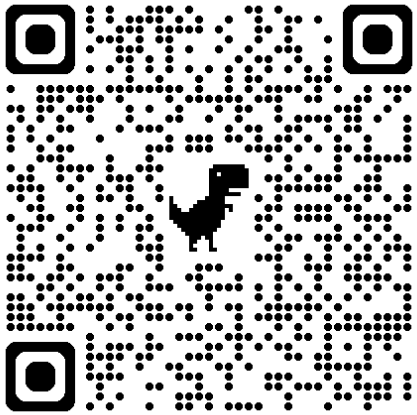
Project A: Looped transformers for road segmentation
Project B: Single-channel speech enhancement



Project A: Think Again, Segment Better

Looped transformers for road segmentation

Teams of three



What is semantic segmentation?

The model gives every pixel a label, rather than giving one label to the whole image.

Satellite image



Land-cover annotation



Input image x

$$\text{shape}(x) = 3 \times H \times W$$

3 RGB channels, image height H, and image width W

Output logits y

$$\text{shape}(y) = C \times H \times W$$

C is the number of classes. Each pixel receives C class scores.

Project A is binary road versus non-road. Its final head may store one road logit per pixel, then threshold the road probability at 0.5.

Our task: road versus non-road. Yellow pixels mark roads in this figure.

Images: Wang et al., LoveDA (NeurIPS 2021), Fig. 1; CC BY-NC-SA 4.0. Crops from the project brief.

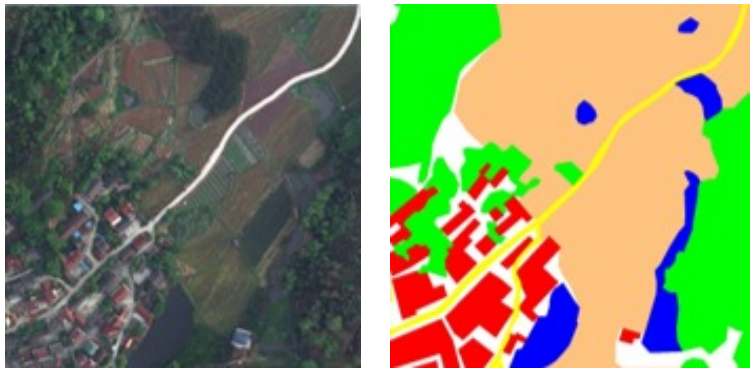
The dataset: urban and rural roads

Urban



Dense buildings and varied road widths

Rural



Fields, vegetation, and narrow roads

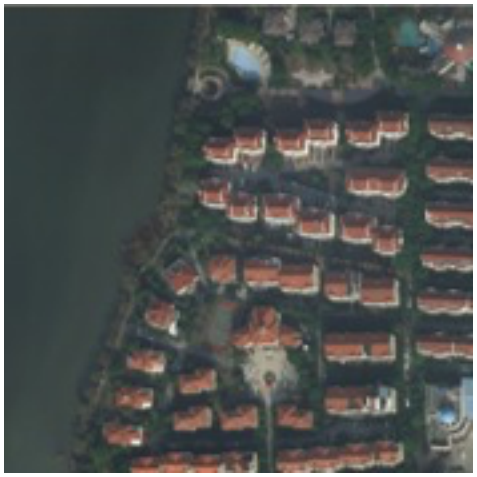
Course tiles: 1024 × 1024 RGB pixels, approximately 0.3 m per pixel.

Images: Wang et al., LoveDA (NeurIPS 2021), Fig. 1; CC BY-NC-SA 4.0. Crops from the project brief.

Why road segmentation is hard



Shadows and obstructions



Roads beside similar bright surfaces



Thin roads in vegetation

Moreover…

Short gap: breaks a route even when area overlap remains high.

False bridge: connects streets that should remain separate.

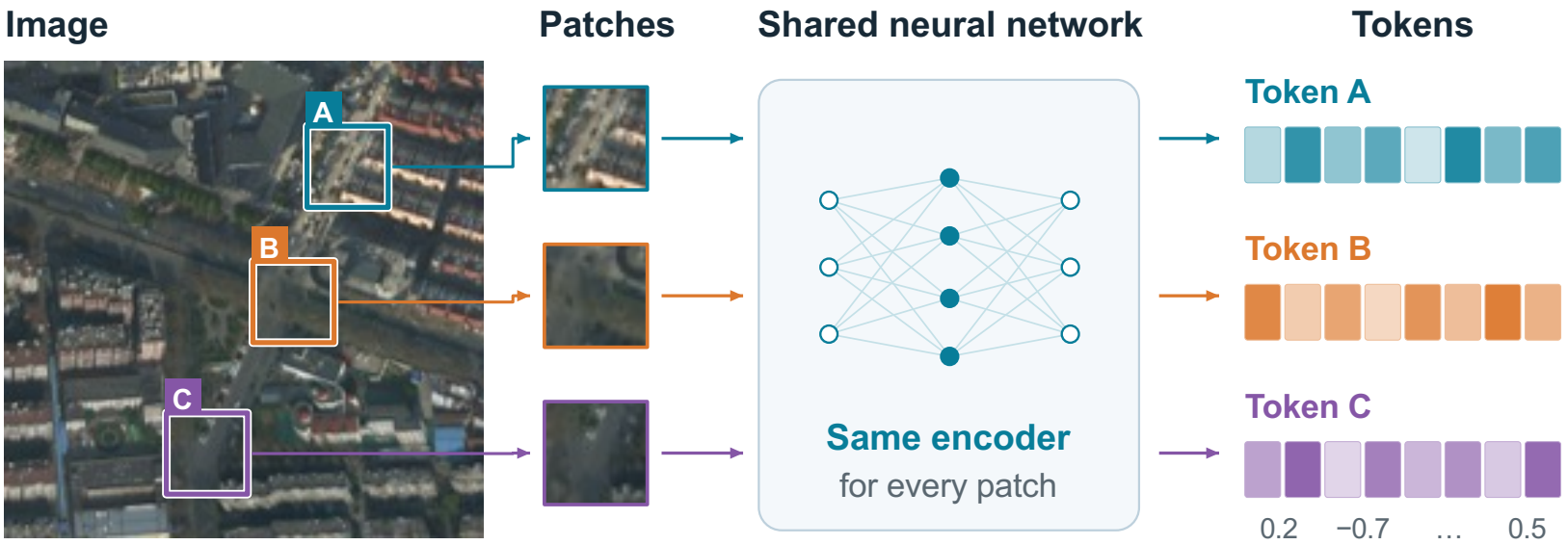
Thick boundary: can swallow a sidewalk or nearby structure.

Project hypothesis: repeated updates can combine image evidence with the current mask to repair some errors.

Images: Wang et al., LoveDA (NeurIPS 2021), Fig. 1; CC BY-NC-SA 4.0. Crops from the project brief.

What is a token?

A neural network turns each image patch into a vector of learned features.



Patch location is added so the model can keep track of where each token belongs.

Image: Wang et al., LoveDA, Fig. 1 (CC BY-NC-SA 4.0). Vector colors and numbers are illustrative.

What is a transformer?

Attention lets each token gather useful information from other tokens.

For road token B: → the model learns how much context to use.



Every token repeats this process.
Position information preserves location.

Illustrative weights for one attention head. Residual connections and normalization are omitted from this view.

What is a transformer?

One attention head builds queries, keys, and values with learned linear layers.

$$Q = X \cdot W_Q, \quad K = X \cdot W_K, \quad V = X \cdot W_V$$

$$\text{Attention}(X) = \text{SoftMax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) \cdot V$$

$X \in R^{N \times d}$	N token vectors, each with d features.
W_Q, W_K, W_V	learned matrices in three linear layers.
Q, K, V	**Q, J, V:** query, key, and value matrices.

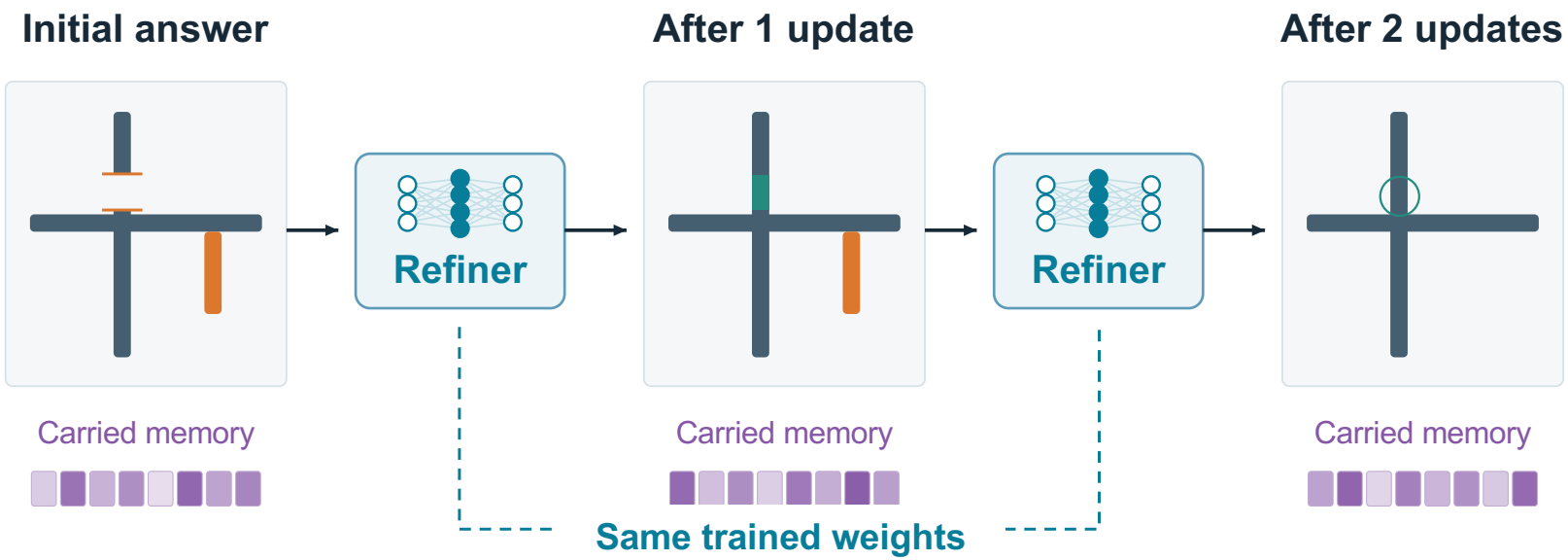
$Q \cdot K^T$	pairwise query-key similarity scores.
$\text{SoftMax}()$	turns each row into weights that sum to one.
	Finally: multiplying by V produces the mixed features.

Most papers call the key matrix K. The project brief writes J so K can mean the number of refinement passes through the shared block.

Source: Project A brief, p. 2, Eq. 1.

Recursive refinement

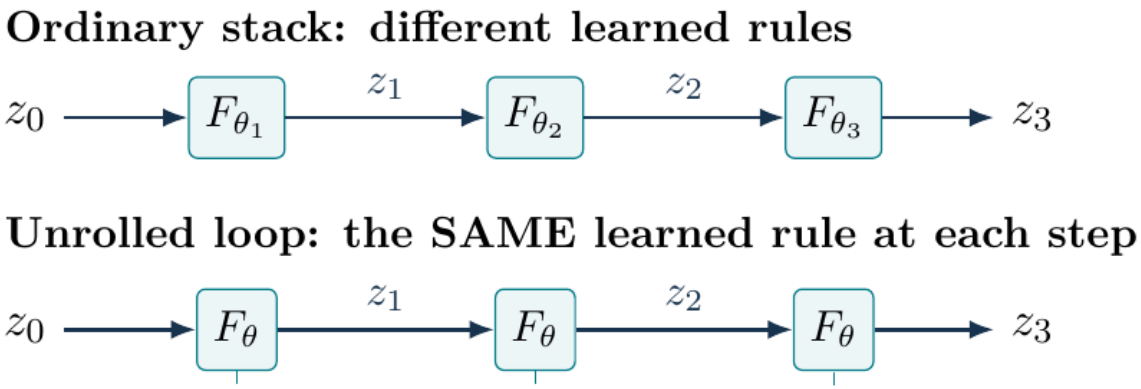
Revisit the current answer using the same learned update.



State changes at each call. More calls cost more computation.

Schematic predictions, not model results. A gap may close or an extra branch may disappear; later updates can also hurt.

What is a looped transformer?



Ordinary stack: a different transformer block is learned at each depth.

Looped transformer: the same block runs several times.

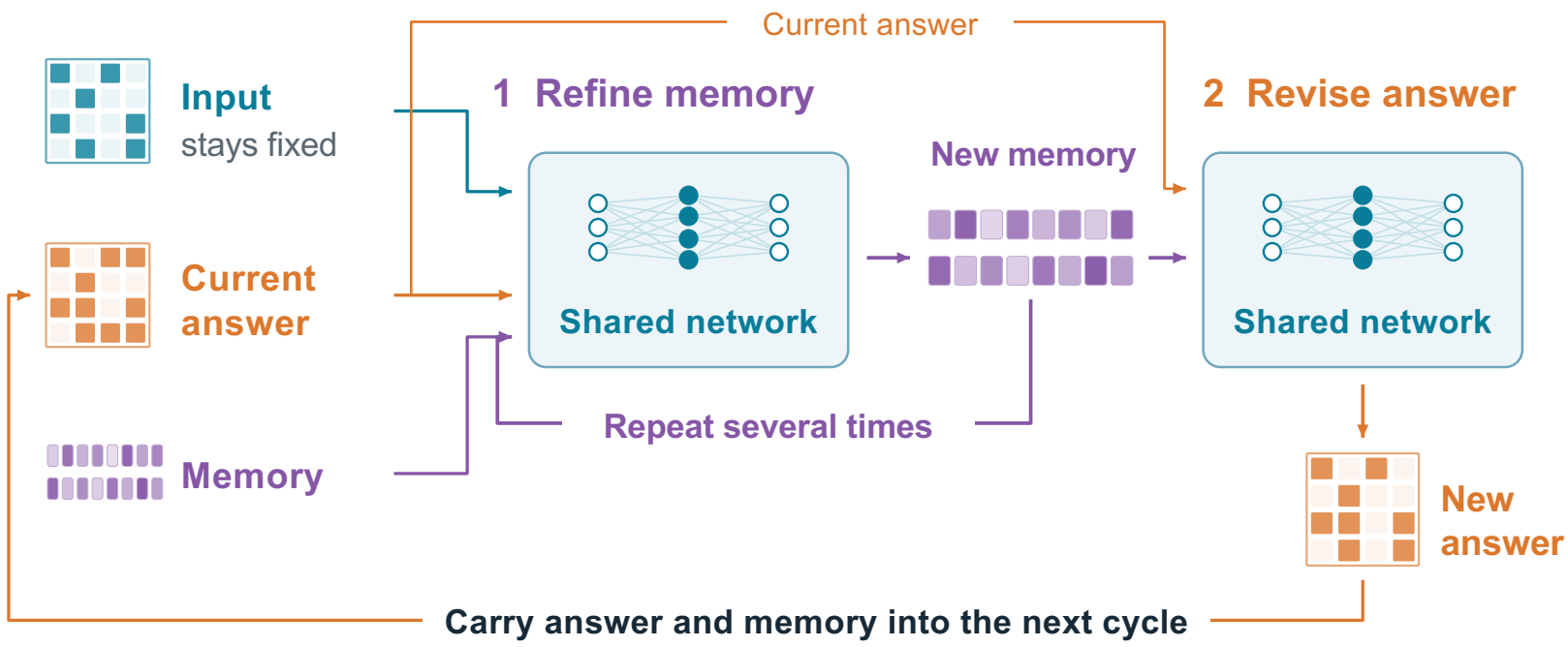
Changing state: the prediction and memory can change on every pass.

Project question: when does another pass repair the road mask, and when does it make the mask worse?

Source: Project A brief, Fig. 2.

TRM: answer and memory

Keep the problem fixed. Refine memory, then revise the answer.



TRM concept. The road project adapts this idea with a simpler loop of exactly K updates.

One image, two changing states

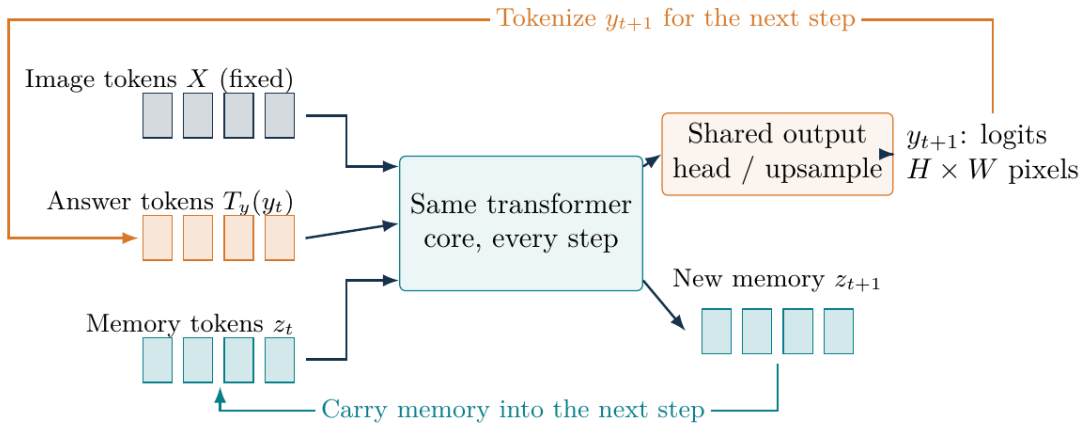
K : number of refinement passes through the shared transformer block.

Frozen start: $f = E(x)$, $y_0 = H_0(f)$.

Memory: $z_0 = I\phi(f, y_0)$.

Shared step: $(z_{t+1}, y_{t+1}) = R\theta(f, z_t, y_t)$.

Output: $pK = \sigma(yK)$, thresholded at 0.5.



Reuse the same core and adapters at every step; carry z and feed back y.

K = 0 means no student update and returns the untouched staff baseline. Ranking runs exactly K updates.

No independent trainable U-Net or CNN segmentation bypass is allowed.

Source: Project A brief, Fig. 3 and model contract on p. 3.

Feasible design choices

Choice	Reasonable starting point	One research direction
State	Low-resolution feature grid	Multiscale grids or learned memory tokens
Attention	Local or window attention	Coarse global context, cross-attention, or sparse links
Output update	Small correction to logits	Replacement logits or gated, uncertainty-aware updates
Feedback	Previous logits plus persistent state	Probability or uncertainty embeddings; feedback ablations
Training	Short unroll with intermediate supervision	Random loop count K, curriculum, truncated gradients, or topology loss

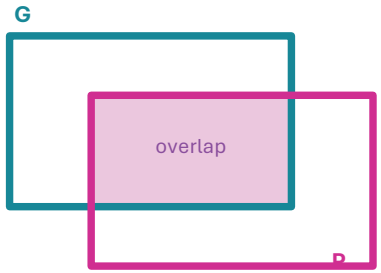
K is the number of shared refinement passes. Ranking requires a fixed exact K; adaptive halting is a separate experiment.

Source: Project A brief, design choices on p. 4.

How segmentation metrics see different errors

Dummy masks for explanation only. G is ground truth and P is the prediction.

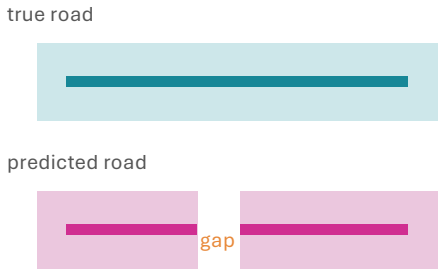
Dice and Road IoU



$$\mathrm{IoU} = \frac{|P \cap G|}{|P \cup G|} \qquad \mathrm{Dice} = \frac{2|P \cap G|}{|P| + |G|}$$

Both compare filled road area. Dice and IoU differ only in normalization. Project A ranks with Road IoU.

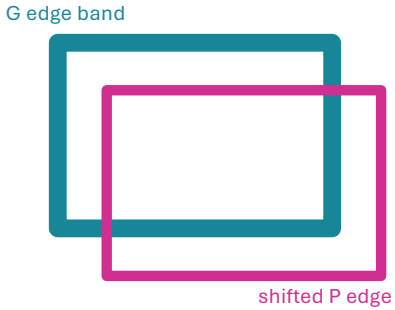
clDice



Skeletonize each mask to a one-pixel centerline, then check whether each centerline stays inside the other road mask.

A short gap may change little road area but reduces centerline coverage.

Boundary IoU



Form a narrow band around each foreground edge, then compute intersection over union only on those bands.

A shifted border can keep most road area but loses boundary agreement.

Definitions follow Project A brief, pp. 5-6; diagrams are editable dummy examples.

Three complementary segmentation metrics

50%

Road IoU

road area overlap

30%

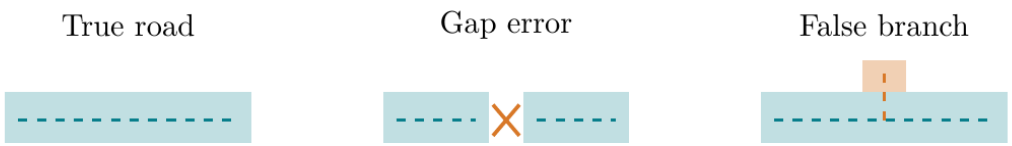
clDice

centerline coverage

20%

Boundary IoU

edge alignment



$$A(D, K) = 0.50\ IoU(D, K) + 0.30\ clDice(D, K) + 0.20\ BIoU(D, K)$$

D is the evaluation image set. K is the number of shared refinement passes.

Evaluate original-resolution masks; label 3 is road, label 0 is ignore, and the threshold is 0.5.

Pool integer counts across the evaluation set before division; also report urban and rural domains separately.

A strong score does not prove routability or reasoning, so failure cases and controlled interventions remain required.

Source: Project A brief, pp. 5-6; schematic from Fig. 4.

Leaderboard score and stress testing

Evaluation images	Refinement steps	Score weight
Ordinary	1	20%
Ordinary	2	20%
Ordinary	4	40%
Stress	4	20%



Score = 100[0.20 A(ordinary, 1) + 0.20 A(ordinary, 2) + 0.40 A(ordinary, 4) + 0.20 A(stress, 4)]

Loop count K: K is the number of shared refinement passes. Ranking uses K = 1, 2, 4; diagnostics also report K = 0, 6, 8.

Stress suite: one deterministic shadow, occlusion, or blur per tile; the ground-truth label stays unchanged.

Leaderboard: proposed 30% public feedback and 70% private ranking, with two submissions per team at most.

Required reporting: urban/rural and ordinary/stress breakdowns, parameters, latency, memory, and GPU-hours.

Source: Project A brief, pp. 7-8; score allocation from Fig. 6.

Proposed Project

Can a compact shared refiner improve a frozen road segmenter by revisiting the same prediction several times?

- **Idea:** use a small shared transformer to revise a road segmentation.
- Start from a **common frozen segmenter**. Train your refiner on top of its features and initial prediction.
- Reuse the core and all adapters. Carry memory and feed back the current answer.
- **Research question:** when do extra iterations repair mistakes, and when do they create new ones?

Source: Project A brief, pp. 1-4 and 10.

Setup

Data and baseline

- Course package: 1024 x 1024 RGB satellite tiles at about 0.3 m per pixel, with urban and rural scenes.
- Binary road versus non-road segmentation; raw label 3 is road and label 0 is ignore.
- Reproduce the frozen staff baseline and save both correct and failed predictions.
- Start with crop training and a 1/16-resolution recurrent state.

Implementation and resources

- Use one small shared transformer and train one checkpoint for K = 1, 2, 4 refinement passes.
- Proposed cap: at most 48 GiB on single L40S.
- Read Universal Transformers, Tiny Recursive Models, Recurrent Scene Parsing, clDice, and Boundary IoU.
- Implement the exact-K adapter, which runs the shared block exactly K times, and submit model code & checkpoint for offline evaluation.

Release status: Project A is an instructor-review draft. Staff must publish the final data package, frozen interface, split manifests, hard caps, and confirmed deadlines before launch.

Source: Project A brief, pp. 1, 4, 8-9, and 12.

Setup: papers and code

- **Main paper:** Less is More: Recursive Reasoning with Tiny Networks (TRM).
- **Background:** Universal Transformers for shared weights across depth.
- **Earlier visual inference:** Recurrent Scene Parsing for prediction feedback.
- **Code:** Official TRM repository for the original algorithm; course starter for the road task.
- **Staff release:** data package (ready on Wed), frozen baseline, runner and evaluator. (ready at mid)

The brief's companion demo uses random weights and synthetic images; it is not a trained road segmenter.

Setup: data and labels

Raw label	Meaning in this project
3	Road
1, 2, 4, 5, 6, 7	Non-road
0	Ignore / no-data



Road prediction: probability at least 0.5; ignore no-data pixels.

- Use the course package and its fixed split manifests.
- Train on training data; select settings on valid data.
- Use only permitted data and models; final evaluation labels stay with staff.

Images: Wang et al., LoveDA (NeurIPS 2021), Fig. 1; CC BY-NC-SA 4.0. Crops from the project brief.

Setup: a feasible first model

- Reproduce the frozen baseline and inspect its mistakes.
- Start with a **1/16-resolution state**, local attention, and a small shared transformer.
- Train on crops; overfit a tiny training subset to check the pipeline.
- Supervise intermediate predictions and train a first model for 1, 2, and 4 updates.
- Measure quality, runtime and memory before adding complexity.

Proposed targets: one L40S; ≤5M core / ≤10M total trainable parameters; ≤32 or 48 GiB training memory.

Targets are pending a staff pilot. Final caps, GPU-hours, tiling policy and timeout will be announced.

Expectations

Required evidence

Show the frozen baseline and the same checkpoint at K = 1, 2, 4, 6, 8 refinement passes against both quality and measured time.

Run state-reset and feedback-removal interventions.

Train a matched one-step non-recurrent control; compare an untied stack if feasible.

Include one hypothesis-driven ablation, failures, domain and stress breakdowns, and uncertainty.

Deliverables and understanding

Method diagram, parameter table, original report and slides, code, one checkpoint, pinned environment, and run manifest.

Exact prefix consistency and the same parameter objects at every recurrent step.

Contribution log with an owner and reviewer for each major experiment.

Every member explains one update, the metrics, tensor shapes, and the expected effect of each control.

Milestones: Sep 28 pitch, Nov 2 proposal and midway update, Dec 14-18 final presentation and submission.

Source: Project A brief, pp. 10-11.

Expectations: experiments

- **Iteration curve:** baseline and the same checkpoint at 1, 2, 4, 6, and 8 steps; quality versus steps and time.
- **Memory and feedback:** reset the state; separately replace feedback with the initial prediction.
- **Fair control:** train a one-step model with a similar parameter count.
- **One design ablation:** test a focused hypothesis by changing one major factor.
- **Error analysis:** show successes, failures, and urban/rural and stress behavior.

A well-executed negative result can support an excellent project. Explain what the experiments establish.

Expectations: deliverables and milestones

When	Expected work
Before November 2	Suggested early check: labels, baseline, one update, memory use
November 2	Proposal + midway update: question, diagram, initial results, plan
December 14–18	Final presentation, report, code and weights; exact time TBC

- Submit one nominated checkpoint, documented code and a pinned environment.
- Include the required controls, intermediate masks, compute costs and contribution log.
- Teams of three; every member should explain the full pipeline.

Dates follow the brief and syllabus. No project assessment is due during the December 11–13 study period.

Expectations: what makes a strong project?

Proposed project rubric	Points
Question and proposal	10
Midway work	15
Final research report	30
Reproducible implementation	15
Presentation and individual understanding	15
Leaderboard component	15

A focused claim.
Fair comparisons.
An honest explanation of the failures.

First question: what error do you expect a second look to fix?

End of Project A

Questions?

